

缓存架构方案

架构组

目录

技术方案报告：分布式缓存架构设计 3

 摘要 3

 1. 背景与问题 3

 1.1 现状 3

 1.2 目标 3

 2. 技术选型 3

 2.1 对比分析 3

 2.2 一致性哈希 3

 3. 架构设计 4

 3.1 整体架构 4

 3.2 缓存更新策略 4

 4. 性能预估 5

 4.1 命中率模型 5

 4.2 延迟分布预测 5

 5. 风险与缓解 5

 5.1 缓存击穿 5

 5.2 缓存雪崩 5

 6. 实施计划 6

 7. 结论 6

技术方案报告：分布式缓存架构设计

版本： v1.2

作者： 架构组

日期： 2025-05-12

摘要

本报告描述了一套面向高并发读场景的分布式缓存架构方案。方案采用多级缓存策略，结合一致性哈希算法实现节点扩缩容，目标是在保证数据一致性的前提下，将核心接口的 P99 响应时间控制在 10ms 以内。

1. 背景与问题

1.1 现状

目前生产环境存在以下问题：

- 核心读接口直接访问数据库，高峰期 QPS 达 50,000，数据库连接池接近上限
- 部分热点数据被反复查询，CPU 利用率超过 80%
- 无缓存预热机制，每次服务重启后存在冷启动性能抖动

1.2 目标

- P99 响应时间：≤ 10ms（当前 P99 约 85ms）
- 缓存命中率：≥ 95%
- 支持水平扩展，单次扩容不影响线上服务

2. 技术选型

2.1 对比分析

方案	优点	缺点	适用场景
Redis Cluster	成熟稳定，支持持久化	内存成本高	通用缓存
Memcached	简单高效，延迟极低	不支持复杂数据结构	简单 KV
本地缓存	零网络延迟	多实例数据不一致	只读配置
多级缓存	命中率最高	复杂度高	高并发读

结论： 采用「本地缓存 + Redis Cluster」二级缓存架构。

2.2 一致性哈希

使用虚拟节点的一致性哈希算法管理缓存节点：

```

import hashlib
from bisect import bisect, insort

class ConsistentHash:
    def __init__(self, nodes: list[str], replicas: int = 150):
        self.replicas = replicas
        self.ring: dict[int, str] = {}
        self.sorted_keys: list[int] = []
        for node in nodes:
            self.add_node(node)

    def _hash(self, key: str) -> int:
        return int(hashlib.md5(key.encode()).hexdigest(), 16)

    def add_node(self, node: str) -> None:
        for i in range(self.replicas):
            virtual_key = self._hash(f"{node}:{i}")
            self.ring[virtual_key] = node
            insort(self.sorted_keys, virtual_key)

    def get_node(self, key: str) -> str:
        h = self._hash(key)
        idx = bisect(self.sorted_keys, h) % len(self.sorted_keys)
        return self.ring[self.sorted_keys[idx]]

```

3. 架构设计

3.1 整体架构

```

请求
↓
API 服务（多实例）
↓
L1: 本地 Caffeine 缓存 (TTL: 30s, 最大 10,000 条)
    ↓ Miss
L2: Redis Cluster (TTL: 5min, 3主3从)
    ↓ Miss
数据库 (MySQL 主库)
    ↑ 回填 L2 → L1

```

3.2 缓存更新策略

采用 Cache Aside 模式：

1. 读操作：先查 L1 → L2 → DB，逐级回填
2. 写操作：先更新 DB，再删除 L2 和 L1 缓存
3. 使用 延迟双删 解决并发写的脏数据问题

```
public void updateUser(User user) {  
    // 1. 删除缓存  
    cache.delete(cacheKey(user.getId()));  
  
    // 2. 更新数据库  
    userRepository.save(user);  
  
    // 3. 延迟 500ms 再次删除 (防止并发读写导致脏缓存)  
    scheduler.schedule(  
        () -> cache.delete(cacheKey(user.getId())),  
        500, TimeUnit.MILLISECONDS  
    );  
}
```

4. 性能预估

4.1 命中率模型

根据历史访问日志分析：

- 热点数据 (Top 1%) 占 80% 的访问量
- L1 缓存容量 10,000 条，覆盖约 60% 的 QPS
- L2 命中剩余 40% 中的 90%，综合命中率 $\geq 96\%$

4.2 延迟分布预测

层级	命中率	延迟	加权延迟
L1 本地缓存	60%	~0.1ms	0.06ms
L2 Redis	36%	~1ms	0.36ms
数据库	4%	~50ms	2ms
综合 P99	—	—	≈ 8ms

5. 风险与缓解

5.1 缓存击穿

风险：热点 key 过期瞬间，大量请求穿透到数据库。

缓解：– 使用 Redisson 分布式锁，只允许一个请求重建缓存 – 对热点数据使用永不过期策略，后台异步刷新

5.2 缓存雪崩

风险：大量 key 同时过期，数据库压力激增。

缓解：– TTL 加随机抖动： $TTL = base_ttl + random(0, base_ttl * 0.2)$ – 服务降

级：缓存全部失效时，限流保护数据库

6. 实施计划

1. **第一阶段**（第 1-2 周）：搭建 Redis Cluster，迁移现有缓存逻辑
2. **第二阶段**（第 3 周）：引入 L1 本地缓存，接入监控告警
3. **第三阶段**（第 4 周）：压测验证，灰度上线 20% 流量
4. **上线**（第 5 周）：全量切换，持续观察 7 天

7. 结论

本方案通过二级缓存架构，预期将 P99 响应时间从 85ms 降低到 8ms，缓存命中率达到 96%，同时保持系统的可扩展性和数据一致性。方案已在测试环境验证，建议按计划推进实施。